



# **COS6012-B: Concurrent and Distributed Systems**

## **Java Synchronisation Utilities**

Dr Daniele Scrimieri

# Outline

---

- Last session – basic thread synchronization using:
  - **synchronized** keyword
  - **Lock** interface and its implementation class `ReentrantLock`
  - **Condition** class
- This session:
  - Introducing the high-level utilities
  - **Semaphore** class in Java
  - Practical tips on solving common synchronization problems

# Thread Synchronization Utilities

---

- **Semaphore** class: a counter that controls the access to one or more shared resources.
- **CountDownLatch** class: mechanism that allows a thread to wait for the finalization of multiple operations.
- **CyclicBarrier** class: allows the synchronization of multiple threads in a common point.
- **Phaser** class: controls the execution of concurrent tasks divided in phases. All the threads must finish one phase before they can continue with the next one.

# Thread Synchronization Utilities

---

- **Exchanger** class: provides a point of data interchange between two threads.
- **CompletableFuture** class: provides a mechanism where one or more tasks can wait for the finalization of another task that will be explicitly completed in an asynchronous way in future.

# Semaphore class

---

- A counting semaphore.
- A semaphore maintains a set of permits.
- Each **acquire()** blocks if necessary until a permit is available, and then takes it.
- Each **release()** adds a permit, potentially releasing a blocking acquirer.
- However, no actual permit objects are used; the Semaphore just keeps a count of the number available and acts accordingly.
- Semaphores are often used to restrict the number of threads than can access some (physical or logical) resource.

# Semaphore class - methods

---

- `public Semaphore(int permits [,boolean fair])`  
fair = true semaphore will guarantee FIFO
- `public void acquire([int permits])` throws `InterruptedException`
- `public void release([int permits])`
- `public void acquireUninterruptibly()`
- `public boolean tryAcquire([int permits], [long timeout], [TimeUnit unit])` throws `InterruptedException`

# Semaphore class - methods

---

- `public int drainPermits()`
- `public boolean isFair()`
- `boolean hasQueuedThreads()`
- `protected Collection<Thread> getQueuedThreads()`
- `public final int getQueueLength()`

# Example: Bike rental

---

A semaphore is like a bike rental place, with a fixed number of bikes. If a bike is available the client can take it, otherwise must wait. When a client returns their bike then someone else can take it.

```
public class BikeRental {  
    private final Semaphore semaphore;  
    public BikeRental(int size) {  
        semaphore = new Semaphore(size);  
    }  
    public void getBike() {  
        [...]  
    }  
}
```

- BikesRentalMain
- Client
  - wants a bike
- BikeRental
  - ✓ semaphore
  - ✓ getBike()



```
public void getBike() {  
    try {  
        semaphore.acquire();  
        Long duration = (long) (Math.random() * 10);  
        System.out.println(Thread.currentThread().getName() +  
            " using bike for " + duration + " seconds [. . .] "  
            + semaphore.availablePermits());  
        Thread.sleep(duration);    TimeUnit.SECONDS.sleep(duration);  
    } catch (InterruptedException e) {  
        [. . .]  
    } finally {  
        System.out.println(Thread.currentThread().getName() + [...] +  
            + semaphore.getQueueLength() + " clients waiting!");  
        [...]   
        semaphore.release();  
    }  
}
```

```
public class Client implements Runnable {  
    private final BikeRental bikeRental;  
  
    public Client(BikeRental bikeRental) {  
        this.bikeRental = bikeRental;  
    }  
  
    @Override  
    public void run() {  
        System.out.println(Thread.currentThread().getName()  
            + " going rent a bike");  
        bikeRental.getBike();  
        System.out.println(Thread.currentThread().getName()  
            + " finished! ");  
    }  
}
```

```
public class BikesRentalMain {  
  
    public static void main(String args[]) {  
  
        int noThreads = 10;  
        int noBikes = 4;  
        BikeRental bikeRental = new BikeRental(noBikes);  
  
        Thread thread[] = new Thread[noThreads];  
        for (int i = 0; i < noThreads; i++) {  
            thread[i] = new Thread(new Client(bikeRental),  
                                   "Thread " + i);  
        }  
  
        for (int i = 0; i < noThreads; i++) {  
            thread[i].start();  
        }  
    }  
}
```

# Sample output

---

Thread 0 going rent a bike  
Thread 1 going rent a bike  
Thread 6 going rent a bike  
Thread 4 going rent a bike  
Thread 2 going rent a bike  
Thread 5 going rent a bike  
Thread 0 using bike for 6 seconds, available bikes now: 0  
Thread 6 using bike for 3 seconds, available bikes now: 0  
Thread 1 using bike for 4 seconds, available bikes now: 0  
Thread 8 going rent a bike  
Thread 4 using bike for 7 seconds, available bikes now: 0  
Thread 3 going rent a bike  
Thread 7 going rent a bike  
Thread 9 going rent a bike  
Thread 6 there are 6 clients waiting in the queue!  
Thread 6 is returning the bike  
Thread 6 finished!  
Thread 2 using bike for 9 seconds, available bikes now: 0  
Thread 1 there are 5 clients waiting in the queue!  
Thread 1 is returning the bike  
Thread 1 finished!  
[...]

Is the printed number of  
available bikes correct?

# More on Semaphore class...

---

- **Constructor:** Semaphore(int permits [,boolean fair])
  - The default mode is **non-fair mode**: when a resource is released, one of the waiting threads is selected and is given it; however, it's selected without any criteria.
  - Fair mode, on the other hand, changes this behavior and selects the thread that has been waiting for the longest period of time.
- **acquireUninterruptibly():** the acquire() method, when the internal counter of the semaphore is 0, blocks the thread until the semaphore is released. During this period, the thread may be interrupted; if this happens, the method will throw an InterruptedException exception. This version of the acquire operation ignores the interruption of the thread and doesn't throw any exceptions.

# More on Semaphore class...

---

- **tryAcquire()**: This method tries to acquire the semaphore. If it can, it returns the **true** value. But if it can't, it returns **false** instead of being blocked and waiting for the release of the semaphore. It's programmer's responsibility to take correct action based on the return value.
- **tryAcquire(long timeout, TimeUnit unit)**: This method is equivalent to the previous one, but it waits for the semaphore for the period of time specified in the parameters. If the period of time ends and the method hasn't acquired the semaphore, it will return **false**.

# A few more tips on Java synchronisation

---

- **synchronized** methods vs **synchronized static** methods
- **Synchronized** for blocks: **this** vs other **object**
- **synchronized** vs **performance**
- **Recursive** calls and **synchronized** methods
- Solving **IllegalMonitorStateException**

# synchronized vs synchronized static methods

---

- All the **synchronized** statements (used on methods or blocks of code) use an object reference.
- Only **one thread** can execute a method or block of code protected by the **same object reference**.
- When you use the **synchronized** keyword with a method, the object reference is implicit (**own** object, represented by the **this** keyword).
- When you use the **synchronized** keyword in one or **more methods** of an object, **only one execution thread** will have access to **all these methods**.
- If another thread tries to access any method declared with the **synchronized** keyword of the same object, it will be suspended until the first thread finishes the execution of the method.



# synchronized vs synchronized static methods

---

- Every method declared with the **synchronized** keyword is a critical section, and Java only allows the execution of one of the critical sections of an object at a time. In this case (non-static methods), the object reference used is the **own** object, represented by the **this** keyword.
- **static** methods have a different behavior. In this case, the object reference used is the **class** object.
- Only **one execution thread** will have access to **one of the static synchronized methods** declared with the **synchronized** keyword.
- But a **different thread** can access other **non-static methods** of an object of that class.
- So, **two threads** can access **two different synchronized methods** if **one is static and the other is not**.
- If both methods change the same data, you can have data inconsistency errors!



# synchronized for blocks: this vs other object

- When you use the **synchronized** keyword to protect a **block of code**, you must pass an object reference as a parameter.
- You can use the **this** keyword or **other object references** as well.
- Normally, these objects will be created exclusively for this purpose. You should keep the objects used for synchronization **private**.
- For example, if you have two independent attributes in a class shared by multiple threads, you must synchronize access to each variable. It wouldn't be a problem if one thread is accessing one of the attributes and the other accessing a different attribute at the same time.
- If you use the **own** object (**this** keyword), you might **interfere with other synchronized code** (as mentioned before, the **this** object is used to synchronize the methods marked with the **synchronized** keyword).

```
synchronized (object) {  
    // statements  
}
```



# synchronized vs performance

---

- The **synchronized** keyword **penalizes the performance** of the application, so you must **only use it** on methods that **modify shared data in a concurrent environment**.
- If you have multiple threads calling a synchronized method, only one will execute them at a time while the others will remain waiting.
- If the operation doesn't use the synchronized keyword, all the threads can execute the operation at the same time, reducing the total execution time.
- If you know that a method will not be called by more than one thread, **no need for synchronized** keyword!
- Anyway, if the class is designed for multithreading access, it should always be correct.

# synchronized and performance

---

- We can use the **synchronized** keyword to protect access to a block of code instead of an entire method.
- We should use the **synchronized** keyword in this way to protect access to shared data, leaving the rest of the operations out of this block and obtaining better performance of the application.
- The objective is to have the **critical section** as short as possible, leaving out the long operations of the block that don't use shared data.
- Also, avoid calling blocking operations (for example, I/O operations) inside a critical section.

# Recursive calls and synchronized methods

---

- You can use **recursive calls** with **synchronized** methods.
- As the thread has access to the **synchronized** methods of an object, you can call other **synchronized** methods of that object, including the method that is being executed.
- It won't have to get access to the **synchronized** methods again!

# How to solve `IllegalMonitorStateException`

---

- This exception, when thrown, indicates that the calling thread has attempted to **wait** on an object's monitor, or has attempted to **notify** other threads that wait on an object's monitor, **without owning the specified monitor**.
- Usually appears when calling `wait()`, `notify()`, or `notifyAll()` methods **outside of a synchronized block**.
- A thread becomes the owner of an object's monitor in one of the following ways:
  - By executing a **synchronized** instance method of that object.
  - By executing the body of a **synchronized** statement that synchronises on the object.
  - For objects of type `Class`, by executing a **synchronized static** method of that class.

# How to solve `IllegalMonitorStateException`

---

- So, we need to make every call to `wait()`, `notify()`, and `notifyAll()` methods only **within** a **synchronized** block or synchronized method!
- `IllegalMonitorStateException` is an unchecked exception and thus, it does not need to be declared in a method's or a constructor's throws clause.

Examples with faulty code, generating the exception and how to fix it:

- <https://examples.javacodegeeks.com/java-basics/exceptions/java-lang-illegalmonitorstateexception-how-to-solve-illegalmonitorstateexception/>  
(examples with `wait()` and `notify()` called from a `main()` method)
- <https://www.baeldung.com/java-illegalmonitorstateexception>  
(example with `wait()` called in a `run()` method)

# Example causing IllegalMonitorStateException

---

```
public class IllegalMonitorStateExceptionExample {  
    public static void main(String[] args) {  
        try {  
            HelperClass.obj.wait();  
        }  
        catch (InterruptedException ex) {  
            System.err.println("An InterruptedException was caught: "  
                + ex.getMessage());  
            ex.printStackTrace();  
        }  
    }  
}
```

Exception in thread "main" java.lang.IllegalMonitorStateException  
 at java.lang.Object.wait(Native Method)  
 at java.lang.Object.wait(Object.java:502)  
 at  
main.java.IllegalMonitorStateExceptionExample.main(IllegalMonitorStateExceptionExample.java:8)



# Solution

---

- In order to deal with the `IllegalMonitorStateException` you must verify that all invocations of the `wait`, `notify` and `notifyAll` methods are taking place only when the calling thread owns the appropriate monitor.
- The most simple solution is to enclose these calls inside synchronized blocks.
- The synchronization object that shall be invoked in the synchronized statement is the one whose monitor must be acquired.

# Solution - main() only

---

```
public static void main(String[] args) {  
    try {  
        Thread waitThread = new WaitingThread();  
        Thread wakingThread = new WakingThread();  
        // Start the execution.  
        waitThread.start();  wakingThread.start();  
        // Wait for all threads to terminate.  
        waitThread.join();   wakingThread.join();  
    }  
    [...]  
}
```

<https://examples.javacodegeeks.com/java-basics/exceptions/java-lang-illegalmonitorstateexception-how-to-solve-illegalmonitorstateexception/>

# Acknowledgments (and resources for further reading)

---

- J. Fernandez Gonzalez (2017), Java 9 Concurrency Cookbook, Packt Publishing

## Online tutorials

- <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- <https://examples.javacodegeeks.com/java-basics/exceptions/java-lang-illegalmonitorstateexception-how-to-solve-illegalmonitorstateexception/>
- <https://www.baeldung.com/java-illegalmonitorstateexception>